

SZYMON ŻYREK

SOFTWARE / SYSTEMS ARCHITECT · SENIOR SOFTWARE ENGINEER

stynkdev@gmail.com | szyrek@stynk.eu | +48 505 784 179 | Tricity, Poland | Remote | Available immediately | LinkedIn | GitHub

ARCHITECTURE LEARNED FROM THE IMPLEMENTATION SIDE

12+ years implementing the transitions architecture is meant to enable - from reconstructing inherited C++/Java EE systems, through integration, selective extraction and platform composition, to distributed systems, production ownership, UX and agentic delivery. I use that accumulated perspective to turn ambiguous business problems into complete systems whose technical complexity disappears behind the right abstraction.

BECOMING BY DOING

DISCOVER · DESIGN · BUILD · OPERATE · EVOLVE

MISYS → FINASTRA

2013-2021 · Gdynia

JUNIOR → EXPERT

●	STAGE 1 Junior → independent engineer		DEBUGGING	LEGACY
			RELIABILITY	
	PROBLEM	Inherited TopOffice and related financial-risk systems arrived with outdated setup notes, unfamiliar repositories, mixed C++/Java EE stacks and little original-team context.		
	INTERVENTION	Built a repeatable investigation workflow around logs, debuggers, stack traces, Git history/bisect, Bash, tests, documentation, source code and finding the right human when needed.		
●	EVIDENCE	Regularly reproduced and fixed failures in systems I had not known existed before receiving the task; grew from local fixes into cross-product ownership.		
	COMPETENCE	Engineering independence is not knowing every technology; it is knowing how to make an unknown system reveal the truth.		
	STAGE 2 Engineer - integration & modernization		INTEGRATION	DISTRIBUTED
			CI/CD	
			IDENTITY	
	PROBLEM	Stabilized legacy products now had to participate in common CI, messaging, SSO and cross-product workflows without rewriting everything that already worked.		
●	INTERVENTION	Added adapters, common contracts, JMS/event flows, OpenID/OIDC integration and deployment standardization; modernized CMR persistence from DataXtend ORM to JPA incrementally while product work continued.		
	EVIDENCE	Helped multiple heterogeneous products move toward common platform contracts while preserving their proven internals and delivery cadence.		
	COMPETENCE	Separate implementation from capability: stable seams, protocols and contracts let very different systems become composable.		
	STAGE 3 Senior - selective extraction & subsystem ownership		SCALING	DISTRIBUTED
			IDENTITY	
●	PROBLEM	Useful legacy capabilities could not scale or integrate cleanly because bottlenecks lived in centralized synchronization, data access or incompatible identity models.		
	INTERVENTION	Extracted scalable/stateless processing around existing calculation engines; built UserManagement plus a shared authentication core in C reused by C++, Java via JNI and Objective-C/GNUstep.		
	EVIDENCE	Owned UserManagement as other teams integrated with it; became a cross-team contact and mentor while the extracted path enabled horizontal scale without rewriting the engine.		
	COMPETENCE	Architecture as constraint reconciliation: change the boundary that blocks the required property, preserve what already works.		
	STAGE 4 Expert - platform perspective		PLATFORM	INTEGRATION
●	PROBLEM	Years of acquisitions had left products, eras and stacks; the business wanted one coherent product instead of exposing each inherited system separately.		
	INTERVENTION	Composed capabilities behind OpenAPI contracts, shared identity/configuration and a Docker Compose integrated environment ; the configurable UI/MVP was MisysBoard , work that evolved into the FusionFabric platform direction.		
	EVIDENCE	Saw CMR/KGR/ALM-era capabilities become a configurable API-driven platform where workflows and dashboards could be assembled above legacy engines.		
	COMPETENCE	A system is an abstraction over capabilities; a platform is a language for composing them into new solutions.		
	Intel Corporation - Firmware Engineer		FIRMWARE	DEBUGGING
●			VERIFICATION	
			DATA	
	PROBLEM	SSD firmware exposed timing, state, memory and hardware constraints normally hidden by managed enterprise runtimes.		
	INTERVENTION	Worked on NAND/storage-driver software in Embedded C and Python test automation , investigating failures across the software/hardware boundary.		
	EVIDENCE	Delivered firmware and automated-verification work inside a specialized SSD development environment.		
	COMPETENCE	Learned firsthand what higher-level abstractions buy - and which hardware/runtime constraints they simplify rather than erase.		

Oct 2021 - Mar 2022 · Gdańsk

THE THREAD SO FAR

Legacy debugging taught me to reconstruct systems I did not yet understand; integration moved the focus to stable boundaries; subsystem ownership and platform composition taught me to change the boundary instead of rewriting the world; firmware made runtime constraints physical.

